



OMNILUME

MASTER PRODUCT ROADMAP & IMPLEMENTATION BLUEPRINT

Replacement edition: the complete dependency-aware plan for evolving the current collaborative room product into a secure, social, multi-purpose platform.

Integrated scope: current remediation, existing roadmap, and the 54 newly requested room, collaboration, management, and intelligence features.

Document purpose: show what exists, what is wrong, what must be built, the exact order of work, how to protect the codebase, and what must be proven before launch.



1. CURRENT STATE

The latest recorded project evidence, separated by evidence level.

READINESS

The core room foundation is substantially remediated, but the full OmniLume vision is not production ready. Missing identity/social/music systems and unverified runtime gates must remain visible instead of being treated as complete.

What OmniLume is

OmniLume is a platform for shared digital spaces. Rooms are collaborative spaces for watching, chatting, studying, files, media, presence, and temporary or permanent lifecycle rules. Personal Chat is separate: it contains friends, General conversations, and persistent groups created through explicit room conversion.

Evidence snapshot

Evidence level	What is known
Implemented in source	Public homepage controls removed; shared room provider; React Player v3 migration; E2EE per-message tolerance; recovery/permanent-room source; Action 10 application-writer cleanup.
Static verification	TypeScript, production build, lint, and diff checks were reported passing; lint warnings were reported as pre-existing.
Database/security verification	Migrations 002-004 and critical anonymous/authenticated RLS checks were previously reported aligned/passing; current 005 and canonical schema still need controlled recheck.
Runtime / multi-user	A to B and B to A chat, reload persistence, Delete for Everyone propagation, production smoke, and independent profile isolation were reported passing. Watch/seek/presence/recovery browser breadth remains open.
Production verification	A production deployment and homepage smoke were recorded, but this roadmap is not a new production audit and does not upgrade missing evidence.

Current code inventory

- Routes: public homepage, login, explore, create-room, room/[id], messages, home, room settings, and internal cleanup endpoint. /room redirects to /explore.
- Room core: src/actions/rooms.ts, members.ts, media.ts, study.ts, recovery.ts, notifications.ts; RoomRealtimeProvider, useRoomSync, useRoomPresence, RoomChat, MediaStage, FilesTab, StudyStage, MembersTab, and RoomNotifications.
- Security and data: Supabase server/browser/SSR clients, src/proxy.ts, 002_rls_lockdown.sql, 003_room_lifecycle.sql, 004_security_remediation.sql, local 005_restore_room_creation_policy.sql, POLICIES.md, and the base schema snapshot.
- Private E2EE: encryption helpers, usePrivateChat, PrivateChat, local device key behavior, Promise.allSettled, and a safe undecryptable-message state.

2. CRITICAL BLOCKERS

Resolve these before calling the expanded product ready.

Priority	Problem	Why it matters	Completion condition
P0	Fresh private-room creation / isolation	An earlier authenticated create-room request returned 403, blocking a clean fresh-room matrix.	Reproduce and trace proxy/session/action/RPC/RLS; fix only if real; prove creator/member/non-member/anonymous/expired cases.
P0	Live database truth	Local migrations and live objects must agree; the recorded state needs current verification, especially local 005 and canonical docs.	Inspect migration list plus policies/functions/triggers/columns; never reset or replay 002/003.
P0	Independent collaboration evidence	Shared-browser tests can leak cookies/storage and do not prove two users.	Two genuinely independent persistent profiles prove chat, Watch, seek, delete, presence, files, and reconnect behavior.
P1	Lifecycle semantics	The final promise is expiry -> 24-hour request window -> approval -> exactly 7-day reopen -> permanent conversion.	Time-controlled disposable-room tests prove authorization, deadlines, cleanup, notification audience, and no resurrection.
P1	Product ambiguity	Requirements conflict on whether converted groups retain room-style features; the master plan also differs on group identity.	Approve one decision record before group schema/UI work; default to preserve data and remove public room username.
P1	Large missing product areas	Google onboarding, profiles, social graph, Personal/General chat, groups, and Music are not complete in the recorded implementation.	Build through the ordered actions and label each evidence level honestly.

Codebase protection rules

- Extend -> improve -> refactor only when needed -> rewrite only when the existing architecture cannot support the requirement.
- Keep one authoritative room realtime path, one authoritative delete path, the existing E2EE protocol, current room URLs, current RLS boundaries, and existing data.
- Do not remove legacy files merely because they look unused. Prove imports, exports, dynamic references, tests, routes, and database dependencies before cleanup.
- Do not add a plaintext fallback, weaken RLS, expose provider tokens, use the browser as the security boundary, or use production data for destructive tests.

3. PRODUCT SEPARATION AND DEPENDENCY MAP

CORE RULE

Rooms and Personal Chat are separate systems. Room membership does not grant access to personal conversations. A room becomes a Personal group only after an explicit authorized conversion.

Dependency flow

Order	Foundation	What it unlocks
1	Foundation and evidence	Safe extension points, status truth, repeatable testing.
2	Identity and profile privacy	Stable users, onboarding, public/private access.
3	Database authorization	RLS, role permissions, relationship and content protection.
4	Room membership and lifecycle	Create/join/leave, roles, expiry, recovery, permanent state.
5	Shared realtime	One event model for Chat, Watch, Files, Study, presence, Music.
6	Core collaboration	Messaging, Watch, files, study, notes, events, room management.
7	Social and groups	Profiles, followers, friends, Personal/General, conversion.
8	Music	Personal uploads/providers and permissioned shared music.
9	AI intelligence	Assistant, summaries, minutes, tasks, room memory after normal systems work.
10	Mobile, performance, release	Reliable cross-device product and production evidence.

Conflicting requirement that must be decided

One specification says converted groups keep room features in a workspace; another says converted groups are chat-first and do not automatically contain Watch, Study, Focus Lock, room presence, or room expiration. The roadmap preserves both possibilities as an explicit decision gate. Until resolved, the safest implementation is: preserve eligible data, create one permanent group, remove the public room username, and expose workspace features only through explicit group permissions.

ACTION 01 - Secure and Stabilize the Foundation

Priority: P0 | Status: PARTIAL | Depends on: Current codebase | Unlocks: Every later action

P0

PARTIAL

Why now?

Security, dependency truth, and the earlier 403 must be understood before adding new tables or routes.

Why not earlier?

The current remediated architecture is the foundation; a rewrite would introduce more unknowns.

Why not later?

Every later feature multiplies the cost of unresolved authorization or schema drift.

Current state

Next.js 16.3.3, React 19, Supabase clients, typed actions, current routes, shared provider, E2EE, and ordered migrations exist.

Existing code/features to preserve

All current routes/actions/providers, existing data, room behavior, E2EE protocol, and user-facing navigation.

What changes

A verified inventory, evidence matrix, current remote state, 403 root cause, and product decision record.

Implementation plan

- Record Git/remote/deployment baseline without altering code.
- Map every sensitive path from UI to server action to database/storage/realtime.
- Classify each feature as source, static, local, multi-user, database, or production verified.
- Create a release evidence folder/checklist without secrets.

Database changes

No broad schema change; only confirmed additive changes after inspection.

Realtime changes

Use the current provider/channel boundaries; do not create a second room connection.

UI changes

Add status/empty/error states and operator evidence, not a parallel product shell.

Security considerations

Never fix availability by weakening RLS or logging secrets.

What could break?

False completion claims, accidental refactor, hidden duplicate path.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- Git status, package scripts, source inventory, tsc, build, lint, diff check, safe smoke probes.

Definition of Done

- All gaps have an owner and evidence requirement.
- No unrelated code is modified.
- Release decisions use the current repository and live evidence.

ACTION 02 - Reconcile Supabase Schema, RLS, Storage, and Canonical Documentation

Priority: P0 | Status: PARTIAL | Depends on: Action 1 | Unlocks: All secure data features

P0

PARTIAL

Why now?

Database authorization protects against direct API/database bypasses and must precede social, groups, music, and AI data.

Why not earlier?

The existing 002-004 remediation is already the intended basis; replaying it risks duplicate policies and history damage.

Why not later?

New data models would inherit inconsistent or stale policy assumptions.

Current state

002, 003, 004, local 005, base schema, and POLICIES.md exist; prior reports recorded 002-004 aligned and critical probes passing.

Existing code/features to preserve

RLS intent, storage path scoping, active/reopened checks, message ownership, service-role cleanup boundary, legacy room_messages table.

What changes

Current live comparison, canonical schema/docs correction, 005 verification, safe indexes/constraints for new domains.

Implementation plan

- Inspect pg_policies, functions, triggers, columns, grants, migration history, and storage policies.
- Verify anonymous, non-member, pending, rejected, former, member, admin, and owner behavior.
- Protect immutable room/message/request identities and relationship pairs with constraints or triggers.
- Update stale canonical policy definitions only after live/effective state is confirmed.

Database changes

Additive tables/columns/constraints/indexes/policies only; no DROP TABLE, TRUNCATE, bulk DELETE, or room_messages deletion.

Realtime changes

Authorize channels by current membership, feature permission, and group membership.

UI changes

Make denied, expired, pending, and permanently available states truthful.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Public SELECT, self-promotion, sender-only update, wrong storage path, dangerous SECURITY DEFINER, expired-room bypass.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- Direct RLS matrix, storage reads, migration list, live catalog inspection, policy diff, no-secret audit.

Definition of Done

- Repository and live security state match.
- No private data is anonymously readable.
- Former members cannot mutate old messages.
- Future developers cannot recreate stale insecure policies.

ACTION 03 - Finish Google Authentication and First-Run Identity

Priority: P0 | Status: MISSING | Depends on: Actions 1-2 | Unlocks: Profiles and social graph

P0

MISSING

Why now?

All later privacy rules need one stable identity and a complete/incomplete onboarding state.

Why not earlier?

OAuth configuration and profile fields depend on the security review and a decision about legacy compatibility.

Why not later?

Social data, provider connections, and private requests cannot be safely attached to an unstable identity.

Current state

Supabase auth, login page, proxy/session refresh, safe next-path behavior, and existing auth records exist; Google onboarding is not complete in the recorded implementation.

Existing code/features to preserve

Existing accounts, session cookies, proxy matcher, login UX, server/browser clients, and direct route protection.

What changes

Google provider/callback, setup gate, profile completion, OAuth config documentation, logout and refresh handling.

Implementation plan

- Configure Google Cloud and Supabase provider URLs for local, preview, and production.
- Load the existing profile after OAuth; route complete users to app and incomplete users to setup.
- Keep next-path redirect internal and safe; do not duplicate profiles.
- Document manual dashboard configuration and environment variables without secrets.

Database changes

Additive profile fields/constraints/RLS; no auth-record deletion.

Realtime changes

Refresh session through existing SSR proxy; do not add broad auth channels.

UI changes

Loading, OAuth error, profile incomplete, logout, and safe redirect states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Account duplication, unsafe redirect, private profile leakage, provider misconfiguration.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- OAuth callback, existing/incomplete profiles, logout/login, session refresh, route protection.

Definition of Done

- A Google user cannot bypass required profile setup.
- Existing complete users are not duplicated.
- Auth/session behavior remains intact.

ACTION 04 - Build Profiles, Privacy, Followers, and Friends

Priority: P1 | Status: MISSING | Depends on: Action 3 | Unlocks: Relationship-gated messaging and discovery

P1

MISSING

Why now?

The social graph decides who can follow, message, or become friends.

Why not earlier?

It requires stable Google identity and profile privacy rules.

Why not later?

Personal/General chat and member actions would otherwise invent inconsistent relationships.

Current state

Profiles exist; full social graph and profile/post privacy are not recorded as implemented.

Existing code/features to preserve

Profile identity, existing public-key behavior, E2EE, and privacy-safe storage.

What changes

Profiles/posts tables or columns, follows, follow requests, friend requests, normalized friendships, relationship UI and RLS.

Implementation plan

- Use directional follows with pending/accepted/rejected/cancelled states and unique pairs.
- Use normalized friend pair ordering and reverse-direction duplicate prevention.
- Accepting friendship safely creates mutual follows; removing friendship does not silently delete follows unless approved.
- Keep requests visible only to the involved users.

Database changes

Additive relationship tables, statuses, constraints, indexes, and least-privilege policies.

Realtime changes

Publish authorized request/friendship events with dedupe.

UI changes

Relationship buttons: follow, pending, accept/reject, add friend, friends green tick, remove.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Self-follow, duplicate requests, accepting another user's request, private list leakage.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Profile setup	New user enters name, DOB, gender, picture, bio, visibility; then reaches app.	Reuse profiles and Supabase storage; add setup route/actions/validation.	RLS, safe defaults, private DOB/gender; depends on Auth.	Incomplete/completed flow, invalid fields, image limits; setup gate is enforced.
Public/private profiles	Public profile is discoverable; private profile limits content and lists.	Profile page and privacy setting; add visibility policy.	Server/RLS, not hidden buttons; no public friend count.	Two-user visibility matrix and privacy switch.
Posts	User publishes posts; visibility follows profile privacy.	Reuse profile identity; add post records/UI.	Owner writes; private reads only for accepted followers/friends.	Public/private post read and write tests.
Lowercase username	Stable lowercase discovery name and safe profile link if approved.	Reuse existing username conventions; add unique normalized constraint.	Prevent impersonation, invalid characters, enumeration concerns.	Case/duplicate/self lookup tests and safe redirect.

How it will be tested

- Three/four-account public/private matrix, RLS direct tests, relationship browser tests.

Definition of Done

- Friends are mutual; follows remain directional.
- Private posts/lists are protected.
- Friend count is never public.

ACTION 05 - Complete Room Identity, Membership, Roles, and Control

Priority: P1 | Status: PARTIAL | Depends on: Actions 1-2 | Unlocks: Managed rooms and safe collaboration

P1

PARTIAL

Why now?

Rooms are the established product anchor; every activity and future conversion depends on correct membership and permission state.

Why not earlier?

The earlier create-room 403 and existing RLS must be traced before adding more controls.

Why not later?

Files, Watch, social member actions, and conversion would otherwise rely on guessed permissions.

Current state

Create/join, public/private rooms, room usernames, roles, membership, and core settings exist; fresh private-room proof and granular features remain.

Existing code/features to preserve

Room URLs, code/link formats, membership approval, owner transfer rules, current activities, lifecycle, storage, shared provider.

What changes

Leave/rejoin, invites, Control Center, granular permissions, ghost/guest identity, lock/rules/welcome, member actions.

Implementation plan

- Trace create/join through proxy, server action, RPC, and RLS; fix the smallest real issue.
- Define owner/admin/member/former/pending/rejected transitions.
- Persist permission and feature settings; enforce each through action, RLS, and realtime.
- Ensure leave removes active membership/presence/subscriptions but not room data.

Database changes

Additive room settings/permission/alias fields or tables, constraints, and policies.

Realtime changes

Shared provider sees membership/permission changes; removed users are unsubscribed.

UI changes

Control Center, member menu, leave confirmation, lock/welcome/rules, dashboard states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Owner lockout, admin escalation, stale membership, ghost identity abuse, API bypass.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Invite link + unique code	Owner/member can share a room link/code; supported formats still join correctly.	Reuse processRoomJoin, room username, get_room_for_join; add validation/display.	Authorization, expiry, private approval, no enumeration.	Public/private code/link tests and expired link denial.
Room roles and permissions	Owner assigns admins and granular permissions; admins act only when allowed.	Reuse members/actions/RLS; add permission matrix.	Server action + RLS + realtime checks; owner always protected.	Owner/admin/member/former matrix.
Room Control Center	One settings area for roles, features, notifications, lifecycle, appearance, and audit.	Reuse room settings/actions/notifications; add organized UI.	Only authorized managers can change settings.	Role access, audit, reload persistence.
Member 3-dot actions	Open profile, follow/friend request, DM/chat request, mute, remove, kick, ban where allowed.	Reuse profiles/social actions and member actions.	Do not reveal private requests or bypass membership.	Owner/admin/member visibility and direct action tests.
Ghost / guest identity	User can appear with a temporary room identity where allowed; account identity stays protected.	Reuse room membership/presence; add scoped alias record.	Abuse controls, owner visibility, no anonymous security bypass.	Alias, leave, ban, reconnect, moderation tests.
Leave / kick / ban / block	Leave cleans membership/presence; owner cannot leave without transfer; managers can remove within permission.	Reuse membership, proxy, provider cleanup.	RLS and server authorization; no data deletion of others.	Leave/rejoin, former member, ban/block tests.
Room rules / welcome / lock	Owner publishes rules and can lock new joins while current access remains explicit.	Reuse room settings and join path.	Rules are room-scoped; lock enforced at join/API.	Locked room, welcome message, join denial.
Room dashboard/home	Room landing area shows current activity, members, features, notices, and lifecycle state.	Reuse room route/provider; add dashboard view.	Only authorized room data; avoid broad queries.	Empty/active/expire d/reopened states.

How it will be tested

- Fresh private-room creation, public/private join, role matrix, leave/rejoin, lock, invite/code, member actions.

Definition of Done

- Creator/member access passes.
- Former/pending/rejected users cannot act as approved members.
- Owner cannot accidentally strand the room.

ACTION 06 - Create the Shared Room Realtime Foundation

Priority: P1 | Status: PARTIAL | Depends on: Actions 2 and 5 | Unlocks: Chat, Watch, presence, files, study, Music

P1

PARTIAL

Why now?

One shared connection prevents duplicate subscriptions and inconsistent room state.

Why not earlier?

It needs stable room identity, authorization, and membership first.

Why not later?

Every future realtime feature would otherwise create another duplicate channel.

Current state

RoomRealtimeProvider, useRoomSync, useRoomPresence, and duplicate RoomChat channel removal exist in source.

Existing code/features to preserve

Provider, room identity, presence, event names, cleanup, and current feature consumers.

What changes

Typed event envelope, authorization guards, dedupe, reconnect/rejoin state recovery, late-join snapshot.

Implementation plan

- Define event ID, actor, scope, type, timestamp, and validated payload.
- Fetch authoritative state on join/reconnect, then subscribe; dedupe by event ID.
- Remove subscriptions on unmount, leave, expiry, and membership removal.
- Keep private keys, plaintext, provider tokens, and large files outside realtime.

Database changes

Only event/state columns if evidence requires them; no sensitive payload tables.

Realtime changes

One room provider; group/social channels will use explicit scopes, not a duplicate room channel.

UI changes

Connecting/reconnecting/stale/permission-lost states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Feedback loops, event floods, stale state, unauthorized injection, ghost presence.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- Two-user chat/delete/presence/Watch/file events; reconnect, late join, duplicate event, removed member.

Definition of Done

- One authorized provider is used.
- Events dedupe and recover.
- No component opens a duplicate room subscription.

ACTION 07 - Build Room Messaging, Notifications, and Social Room Actions

Priority: P1 | Status: PARTIAL | Depends on: Actions 2, 5, 6 | Unlocks: A complete room communication layer

P1

PARTIAL

Why now?

Messaging is the most visible shared workflow and provides the event model for many new room features.

Why not earlier?

It depends on RLS, membership, shared realtime, and profile identity.

Why not later?

Later collaboration, Music, and AI need stable message/activity history.

Current state

Room chat, E2EE private chat, deletion, notifications, and related actions exist; many richer room message features are missing.

Existing code/features to preserve

RoomChat, chat actions, messages schema, Action 5 security, E2EE boundaries, notification infrastructure.

What changes

Edit, pin, thread, mention, announcement, PDF sharing, notification center, DND, and timer notification behavior.

Implementation plan

- Define message types and metadata without mixing room/private/group scopes.
- Use direct DB protections for edit/delete/pin/announcement and safe optimistic UI.
- Use notification rows keyed to user and room, with read state and dedupe IDs.
- Keep room chat plaintext model separate from private E2EE payloads.

Database changes

Additive message metadata, pin/mention/notification tables, indexes, and RLS.

Realtime changes

All events use RoomRealtimeProvider; notification delivery is scoped and deduplicated.

UI changes

Menus, threads, pin panel, PDF card/viewer, mention autocomplete, notification center, DND controls.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Sender ownership bypass, notification privacy, raw crypto logging, duplicate message/event, deleted parent confusion.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Edit/delete own messages	Sender edits or deletes own message; others see the state update.	Reuse chat actions/messages and Action 5 RLS; add edit metadata/UI.	Current approved member + sender + active/reopened room; protect scope fields.	Sender/non-sender /former direct mutation and two-user propagation.
Shared PDFs	A room member shares a PDF; room members see it with realtime notice.	Reuse Files/storage/provider; add PDF metadata/viewer.	Private storage, room membership, feature permission, expiry.	Upload, view, remove, notice, non-member denial.
Announcements	Owner/admin posts a prominent room notice.	Reuse room messages/notifications; add announcement type.	Owner/admin permission, immutable author, room scope.	Role matrix, dismiss/read persistence.
Pinned messages	Authorized manager pins a message; members see pinned list.	Reuse messages and notification center; add pin relation.	Room member read, manager write, no cross-room pins.	Pin/unpin, deletion, refresh, non-manager denial.
Replies / threads	User replies to a message and opens a focused thread.	Reuse message IDs/realtime; add parent/thread metadata.	Parent and child same room/conversation; membership checks.	Nested/reply ordering and delete behavior.
Mentions	User mentions a room member and receives a scoped notice.	Reuse profiles/membership/notifications; add mention parsing.	Only room members, safe display names, no private profile leak.	Mention delivery, muted settings, removed member.
Notification Center	User sees room, mention, timer, PDF, announcement, conversion, and social notices.	Reuse room_notifications and existing notifications action.	Per-user RLS; no unrelated room notifications.	Read/unread, filters, retention, audience tests.
Smart notification controls / DND	Mute room or notification types for a duration or forever, including mentions.	Reuse notification settings; add per-type schedule.	User-owned settings; server honors mute.	Temporary/never/mention-only and reload tests.
Timer-created notification	Starting a shared timer creates a notification for the owner as specified.	Reuse StudyStage, study action, notification center.	Only authorized timer event; no cross-room notice.	Owner notification and duplicate-event test.

How it will be tested

- Two-user send/edit/delete/pin/reply/mention/PDF/timer notice; direct RLS for non-member/former/non-sender.

Definition of Done

- Good messages stay usable.
- Unauthorized users cannot mutate or read.
- All notification audiences are correct.

ACTION 08 - Add Room Activity, Discovery, Events, and Management Tools

Priority: P2 | Status: MISSING | Depends on: Actions 2, 5-7 | Unlocks: Searchable, organized, event-aware rooms

P2

MISSING

Why now?

These features use normal messages, activity, history, and permissions; building them before those foundations would create opaque behavior.

Why not earlier?

They depend on shared realtime, room scope, and data lifecycle.

Why not later?

AI summaries and memory need trustworthy normal history first.

Current state

The master plan mentions activity/history/polls/scheduling; the complete feature set is not recorded as implemented.

Existing code/features to preserve

Room queries, message IDs, provider, notifications, storage, lifecycle, owner/admin roles.

What changes

Search, filters, activity history, bookmarks, polls, countdowns, activity bar, room profiles, backup/export, templates, rule-based organization, trust.

Implementation plan

- Add paginated indexed records and deterministic search/filter paths.
- Define retention and visibility for activity/history/trust.
- Make export scoped, signed, expiring, and auditable.
- Keep auto-organization explainable and reversible; never silently move or delete user content.

Database changes

Additive tables/indexes/RLS; no bulk export of unrelated rooms or data deletion.

Realtime changes

Use shared provider for poll/timer/activity/presence; keep large export jobs server-side.

UI changes

Search/filter UI, timeline, poll cards, countdown bar, bookmarks, export dialog, template picker.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Private room enumeration, export leakage, vote manipulation, trust abuse, unbounded history.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Room search	Search messages, members, files, announcements, or history within the room.	Reuse current room queries/storage metadata; add indexed search fields.	Only current authorized room scope; redact private fields.	Search permissions, empty/no-match, large history.
Activity / room history	Timeline shows joins, settings, files, notices, conversions, and key room events.	Reuse notifications/audit events/realtime; add activity records.	Member-visible vs owner-only fields; retention policy.	Ordering, pagination, removed user, expiry.
Room bookmarks / saved content	User saves a message/file/PDF/clip for later.	Reuse message/file IDs; add user-scoped bookmark relation.	Only content user can currently access; stale references safe.	Save/remove, access loss, refresh.
Polls and voting	Members create and vote in room polls; results update live.	Reuse messages/realtime; add poll/options/votes.	One vote rules, room membership, owner moderation.	Duplicate votes, closed poll, removed member.
Shared countdown / event timer	Room sees synchronized countdown for an event.	Reuse study timer and shared state; add event timer type.	Authorized creator, server timestamp, expired state.	Late join, pause/reset, reconnect.
Live activity bar	Compact bar shows who is watching, studying, typing, uploading, or voting.	Reuse presence/event reducer; add ephemeral activity state.	No sensitive content; current membership only.	Join/leave/reconnect and dedupe.
Room-specific profiles	User can have a room nickname/avatar/bio distinct from global profile.	Reuse profile + membership; add scoped profile fields.	Room members only; Ghost Mode policy explicit.	Different rooms, removal, privacy.
Smart room search and filters	Filter rooms by topic, access, activity, language, duration, and features.	Reuse normal room search; add indexed filter fields before semantic search.	Do not expose private room metadata; AI search comes later.	Public/private filter matrix and performance.
Room backup / export	Authorized owner exports permitted room history/files/metadata.	Reuse storage references, activity, messages, server action.	Owner/admin permission, signed export, expiration, audit.	Disposable export, revoked member, no secret leakage.
Room templates	Create a room from approved settings/features/rules template.	Reuse room creation/settings; add template records.	Owner-owned or public templates; no copied private data.	Template create/apply, permissions, versioning.
Smart auto-organization	Rule-based suggestions group files, notes, clips, and events.	Reuse tags/search/history; add deterministic jobs.	No AI yet; user confirmation and private scope.	Explainable suggestions, undo, no data move without consent.
Room reputation / trust levels	Members earn/receive trust signals that influence guest/moderation controls.	Reuse membership/history/moderation; add scoped trust records.	Avoid discriminatory automation; owner review and appeal.	Change/abuse/removed-member tests.

How it will be tested

- Four-account room matrix, pagination, poll/timer two-user, search private scope, export audit, trust abuse cases.

Definition of Done

- Every tool is permissioned and scoped.
- Normal search/history precede AI search/memory.
- Exports never leak private data.

ACTION 09 - Make the Room Workspace Adaptive and Personalizable

Priority: P1 | Status: PARTIAL | Depends on: Actions 5-8 | Unlocks: Comfortable desktop/mobile room use

P1

PARTIAL

Why now?

The new tools need a consistent place to appear without leaving blank panels or making chat unusable.

Why not earlier?

Layout depends on the actual feature/permission state and shared room shell.

Why not later?

If each feature invents its own layout, the room becomes inconsistent and hard to support.

Current state

Existing room shell and feature mounting behavior exist; center-chat/right-feature and resizable panel requirements remain a target.

Existing code/features to preserve

Room route, current media/study/chat mounting, Focus Lock, shared provider, responsive design tokens.

What changes

Default center chat, active feature plus right chat, resizable panel, presets, owner layout controls, themes.

Implementation plan

- Use one layout state machine and keep feature mounting stable.
- When a feature is disabled, remove its space and expand chat.
- Persist panel size safely per user/room; fall back to mobile layout.
- Validate custom themes and contrast.

Database changes

Additive settings/preset/theme tables or JSON with strict validation and RLS.

Realtime changes

Layout changes emit one scoped settings event; feature state remains in shared provider.

UI changes

Responsive desktop/mobile, resize handle, back/close, no overlap, no blank rectangles.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Hidden feature API bypass, layout lockout, unreadable themes, state reset on navigation.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Custom layout presets	Save default feature/chat arrangement per room or user.	Reuse room shell/layout state; add preset record.	Only authorized preset changes; safe mobile fallback.	Save/load/reset and feature-disabled tests.
Owner-controlled room UI/layout	Owner decides which features are visible, enabled, or emphasized.	Reuse feature permissions/Control Center.	API/realtime denial matches hidden UI; owner/admin granularity.	Member direct API denial and layout refresh.
Room themes / appearance	Room chooses colors, background, density, and contrast-safe appearance.	Reuse room settings and design tokens.	No unsafe contrast, no user-uploaded XSS/background bypass.	Theme permissions, contrast, reset.
Room dashboard/home	A default room home summarizes state before a feature is opened.	Reuse room dashboard from Action 5.	Do not query data user cannot see.	Active/expired/empty and mobile states.

How it will be tested

- Desktop/mobile room screenshots, feature open/close, disabled feature, resize persistence, Focus Lock navigation.

Definition of Done

- Chat is comfortable by default.
- Features do not overlap.
- Owner controls are enforced beyond UI.

ACTION 10 - Complete Watch, Shared Media, and Playback Reliability

Priority: P1 | Status: PARTIAL | Depends on: Actions 2, 6, 9 | Unlocks: Reliable shared watching

P1

PARTIAL

Why now?

Watch is a flagship feature and the strongest test of realtime quality.

Why not earlier?

It needs the shared provider, room permissions, layout, and media lifecycle.

Why not later?

Music and clips can reuse a stable player/state contract afterward.

Current state

React Player v3 source fixes, MediaStage, native currentTime concepts exist; full two-user seek/runtime proof remains open.

Existing code/features to preserve

MediaStage, react-player v3, currentTime, subtitles, provider, storage, room state, permissions.

What changes

Independent play/pause/seek, drift handling, late join/reconnect, queue, now-playing bar, clips.

Implementation plan

- Broadcast local seek once, apply remote seek without rebroadcast, throttle noisy events.
- Fetch authoritative media state for late join/reconnect.
- Support uploaded/external sources honestly; detect unsupported formats.
- Keep queue/clip records within room/group scope.

Database changes

Additive queue/clip metadata only if needed; storage remains private.

Realtime changes

Use one room event reducer for media state; no duplicate Watch channels.

UI changes

Playback controls, drift/reconnect/error state, queue, clips, subtitle/audio controls.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Feedback loop, drift, unsupported source, unauthorized controller, token leak.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Watch foundation	Player loads correct source and subtitles with clear errors.	Reuse MediaStage, react-player v3, native track/currentTime.	No url/getInternalPlayer/config .file/as any; source limits truthful.	Static search, local playback, subtitle/source errors.
Play/pause/seek sync	A and B follow play, pause, resume, and seek in either direction.	Reuse useRoomSync/provider; add throttled local seek + remote suppression.	Authorized room members only; no event flood/loop.	Two independent profiles, repeated seeks, reconnect.
Shared queue / now playing bar	Room sees current track/media and queued items; controls follow permissions.	Reuse player state and future Music model.	Only authorized controller changes queue; no provider token.	Controller/member, late join, deletion.
Clips / highlights	Member saves a bounded media highlight with room context.	Reuse media timestamps, bookmarks, storage.	Source/license/retention; no private media leak.	Create/view/delete, expiry, access denial.

How it will be tested

- A/B and B/A play/pause/seek, repeated seeks, late join, reconnect, subtitles, upload/external, clips.

Definition of Done

- No obsolete player APIs remain.
- Two-user sync has no loop.
- Late join and reconnect recover state.

ACTION 11 - Deliver Collaboration, Voice, Events, Documents, and Room Operations

Priority: P2 | Status: MISSING | Depends on: Actions 5-10 | Unlocks: Rich non-AI collaboration

P2

MISSING

Why now?

These are useful non-AI systems that create the normal history and activity that later intelligence can summarize.

Why not earlier?

They depend on stable realtime, storage, permissions, room layout, and lifecycle.

Why not later?

AI meeting features must come after normal notes, events, activity, and documents are reliable.

Current state

Study, Files, Focus Lock, and some room components exist; voice, documents, events, whiteboard, and operations breadth are not complete.

Existing code/features to preserve

Provider, storage, Study/Focus Lock, notifications, room settings, existing media and membership.

What changes

Notes, voice, screen share, raise hand, docs, cursor presence, events, bookmarks, export, countdown, activity/dashboard, templates, trust.

Implementation plan

- Choose managed providers for voice/screen sharing; keep secrets server-side.
- Use versioned documents and conflict handling where simultaneous edits exist.
- Make events/countdowns server-time based and lifecycle aware.
- Make export bounded, signed, auditable, and unable to restore destroyed data.

Database changes

Additive scoped tables, indexes, RLS, provider secrets, retention metadata.

Realtime changes

Use shared provider/event contract; ephemeral cursor/presence is rate-limited.

UI changes

Feature tabs, document editor, voice controls, hand raise, event cards, export dialog, dashboard.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Call credential leakage, unauthorized document/export, cursor flood, trust abuse, stale timers.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Live shared notes	Members edit a room note and see updates quickly.	Reuse Notes/Study/storage/provider; add versioned note state.	Room/group scope, member permissions, conflict strategy.	Two-user edit/reconnect, history, unauthorized write.
Zoom/Meet-style voice chat	Room members join audio; owner/admin can mute/unmute under permission.	Reuse identity/presence; add managed SFU adapter.	Consent, device permissions, membership, no raw credentials.	Two-user join/leave /mute/reconnect and removal.
Screen sharing	Member shares screen to room when feature/permission allows.	Reuse media stage and call adapter.	Browser permission, active member, no silent capture.	Start/stop, member removal, mobile limitation.
Raise hand	Member signals a request to speak; managers can clear it.	Reuse presence/activity events.	Current member only, no stale state.	Join/leave/reload and clear.
Collaborative document editing	Members work on a shared document with conflict-safe revisions.	Reuse storage, notes, realtime event envelope.	Document RLS, revision history, size limits.	Two-user concurrent edit and rollback.
Shared whiteboard cursor presence	Users see live cursor/pointer labels on a whiteboard.	Reuse presence/provider and whiteboard state.	Ephemeral, current room members, rate-limited.	Cursor joins/leaves/reconnect and flood control.
Room events	Room has scheduled or ad hoc events with reminders and attendance.	Reuse notifications, scheduler, room history.	Owner/admin create; member read; timezone safe.	Create/remind/cancel/expired room.
Room-specific user profiles	Member card uses room alias, role, trust, and activity visibility.	Reuse scoped profile from Action 8.	Never expose global private fields.	Two rooms with different alias/privacy.
Room bookmarks / saved content	Saved messages/files/clips are easy to revisit.	Reuse bookmarks from Action 8.	User-owned relation, current access check.	Access loss and stale reference.
Room backup / export	Authorized owner gets a bounded archive of allowed content.	Reuse export from Action 8 and storage references.	Signed, expiring, audited; excludes destroyed content.	Disposable export and revoked access.
Shared countdown / event timer	All members see one time base and state.	Reuse timer/provider/activity.	Server time and permission; no client-only authority.	Late join, reset, expiry.
Room templates	Room can be started with saved settings/rules/features.	Reuse template/creation path.	No copying private member/content data.	Apply/version/permission.
Room reputation / trust levels	Moderation can use transparent trust signals.	Reuse history/membership/moderation.	No opaque automatic punishment; appeal path.	Abuse and removal tests.
Cross-room personal dashboard	User sees joined rooms, upcoming events, unread activity, and saved content.	Reuse room list, notifications, bookmarks, events.	Only rooms user can access; efficient pagination.	Four-room visibility and expired room filtering.

How it will be tested

- Two-user voice/screen/notes/docs/cursor/timer; RLS; lifecycle; export; mobile and accessibility checks.

Definition of Done

- Every tool is separately permissioned.
- Normal collaboration history is queryable and private.
- AI has a safe foundation but is not yet enabled.

ACTION 12 - Make Expiry, Recovery, Guests, Scheduling, and Permanent Rooms Real

Priority: P0 | Status: PARTIAL | Depends on: Actions 2, 5-11 | Unlocks: Trustworthy lifecycle and retention

P0

PARTIAL

Why now?

Lifecycle promises are data-safety promises. Exact deadlines must be stable before groups, exports, or AI memory.

Why not earlier?

The current lifecycle code and migration are the base; the remaining work is semantics and evidence, not a rewrite.

Why not later?

Later features could accidentally resurrect destroyed data or give old members access.

Current state

Expiry, recovery actions, room-lifecycle helper, cleanup route, permanent requests/notifications, and 004 logic exist; browser/cron proof and some semantics remain open.

Existing code/features to preserve

Active/reopened/permanent checks, cleanup service-role boundary, RLS, preserved data, room access cutoff.

What changes

Exact state machine, guest/lock/schedule records, notification audience, idempotent cleanup and browser verification.

Implementation plan

- Store original expiry, recovery_requested_until, reopened_until, and permanent state without fake future dates.
- Allow owner/admin recovery request only in 24-hour window; pending -> approved/rejected only.
- Approval sets reopened_until = approval time + 7 days and preserves only recoverable data.
- During reopen, any approved member requests permanence; owner/admin reviews; approval clears temporary expiry and creates scoped notifications.
- Cleanup repeatedly and safely handles expired media/rooms without restoration.

Database changes

Additive lifecycle/guest/schedule fields/tables, triggers, RLS, and RPCs; no destructive migration operations.

Realtime changes

Lifecycle/notification events are scoped; removed/expired users lose subscriptions.

UI changes

Clear expired/recovery/reopened/permanent states, guest countdown, schedule, notification wording.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Recovery bypass, former member regain, permanent expiry, wrong audience, non-idempotent cleanup.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Temporary guest access	Owner grants a bounded guest session without turning the room public.	Reuse invite/join, membership, cleanup.	Short expiry, limited permissions, no private data beyond scope.	Guest expiry, revoke, reconnect, RLS.
Guest / temporary identity mode	Guest can use a room alias while account identity remains protected.	Reuse Ghost Mode and room membership.	Abuse/rate limits and owner moderation.	Alias expiry, ban, audit.
Room lock	Stops new joins or changes lifecycle access while preserving current members.	Reuse Control Center and join action.	Server-enforced; no client-only lock.	Direct join denial and unlock.
Scheduled rooms	Room opens/closes or activates features at scheduled times.	Reuse lifecycle, events, cron, room settings.	Timezone, owner auth, cleanup, no stale bypass.	Schedule/retry/expiry/reconnect.
Recovery and permanent room	Expiry -> 24-hour request window -> owner/admin approval -> 7-day reopen -> member request -> owner/admin approval -> permanent.	Reuse recovery.ts, room-lifecycle.ts, 004 RPCs, cleanup, notifications.	RLS, immutable requests, preserved data only, no fake permanent expiry.	Time-controlled disposable-room end-to-end tests.
Permanent notification	Prior legitimate users receive requester identity, approval, permanent and indefinite availability notice.	Reuse room_notifications and provider.	Audience based on authoritative room-user history; unrelated users excluded.	Persistence, realtime, RLS, no fake expiry date.

How it will be tested

- Disposable expiry, 24-hour deadline, 7-day deadline, owner/admin/member matrix, rejection, permanent, notification, cron logs, repeat cleanup.

Definition of Done

- 24-hour recovery window is enforced.
- Reopened access lasts exactly 7 days.
- Permanent rooms have no fake expiry.
- Destroyed data is never restored.

ACTION 13 - Convert Rooms into Persistent Personal Groups

Priority: P1 | Status: PARTIAL | Depends on: Actions 2, 5, 8, 12 | Unlocks: Personal Groups and shared group chat

P1

PARTIAL

Why now?

Conversion is a data migration boundary and must follow a settled lifecycle and Personal Chat model.

Why not earlier?

The group feature surface conflicts in prior requirements; schema must not be locked prematurely.

Why not later?

Existing rooms need a safe path into the social product without duplication or accidental public links.

Current state

convertRoomToGroup and remediation functions exist; current owner-only behavior and final group semantics need confirmation.

Existing code/features to preserve

Originating room reference, eligible data, approved members, owner/admin roles where safe, storage references, no content resurrection.

What changes

Group tables/members/permissions, idempotency constraint, username removal, workspace/back, Personal integration.

Implementation plan

- Authorize owner or explicitly permitted admin; lock duplicate conversions.
- Create one group keyed by originating_room_id; preserve room name; remove room username/public link.
- Exclude pending/rejected/removed/banned members and expired/deleted content.
- Choose chat-only or feature-preserving workspace and document it.

Database changes

Additive group tables, unique originating_room_id, foreign keys, RLS, notifications; no room deletion.

Realtime changes

Emit one conversion event; group events are separate scope from room events.

UI changes

Confirmation/progress/result, group lists, chat-first view, workspace menu, Back.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Duplicate conversion, old member access, public username leak, partial migration.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Room to group conversion	Authorized owner/admin converts once; group appears in Personal Groups and All.	Reuse convertRoomToGroup, membership/data references, storage.	Transactional/idempotent, approved members only, no duplicate group.	Double conversion, partial failure, eligibility, RLS.
Group membership/roles	Approved room members become group members; roles/permissions remain explicit.	Reuse room member roles where safe; add group tables.	Pending/rejected/removed excluded; group RLS.	Member reads/writes, remove, role change.
Group workspace	Three-dot menu opens room-style workspace; Back returns to group chat.	Reuse room shell/provider only if product decision approves.	Group permissions and no public room username/link.	Chat/workspace/back and disabled feature.
Group notifications	Group members see group notices and unread state.	Reuse notification center/provider.	Group-member-only reads.	Member removal and realtime.

How it will be tested

- Disposable room, owner/admin conversion, double click, eligible membership/data, group RLS, workspace decision.

Definition of Done

- One retryable group exists.
- Name remains; room username is removed.
- Only eligible members/data are carried forward.

ACTION 14 - Deliver Personal, General, and Group Messaging

Priority: P1 | Status: MISSING | Depends on: Actions 3-4, 6, 13 | Unlocks: A coherent social inbox

P1

MISSING

Why now?

Relationships and groups now provide the rules for a useful personal messaging experience.

Why not earlier?

It depends on identity, social graph, conversation types, group membership, and E2EE compatibility.

Why not later?

If built earlier, room chat and personal history can be mixed or duplicated.

Current state

Messages page and private E2EE exist; Personal/General/group model is not complete.

Existing code/features to preserve

Existing E2EE protocol, room chat separation, message ordering, device keys, Promise.allSettled, undecryptable UI.

What changes

Conversation types, Personal Groups/All, General requests, group messages, read/unread, friend reclassification, realtime.

Implementation plan

- Model room, private_friend, general, and group scopes explicitly.
- Allow General chat through one-way follow or accepted request; block before acceptance.
- Expose the same history when users become friends; do not create duplicate conversations.
- Keep group membership and friend relationships independent.

Database changes

Additive conversation/group metadata, statuses, unique pairs, member RLS, read-state indexes.

Realtime changes

Authorized personal/group subscriptions with cleanup on membership loss; no room channel reuse.

UI changes

Top-level Personal/General, Groups/All, request inbox, friend/group visual distinction, empty/error states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Cross-scope reads, duplicate chats, pre-accept messages, key leakage, request privacy.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Personal -> Groups	Shows only groups current user belongs to; no individual friends.	Reuse group membership/conversation queries.	Group-member RLS; no room membership shortcut.	Only member groups, unread, reload.
Personal -> All	Combines accepted friends and groups with clear visual distinction.	Reuse friendship/group lists.	No non-friend or unrelated group.	No duplicates, friend/group distinction.
General chat requests	One-way follow permits chat where rules allow; otherwise request -> accept/reject before messaging.	Reuse E2EE/private chat carefully; add request model.	Participant RLS, no pre-accept messages, blocked users.	Request, accept/reject, history, refresh.
Friend member actions	From profile/room menu send friend or DM request; state updates clearly.	Reuse social actions/notifications.	Only involved users; no room-to-personal bypass.	Two-user relationship and request tests.
Personal E2EE	Friend/private messages remain encrypted and tolerate missing keys.	Reuse usePrivateChat, encryption, PrivateChat.	No plaintext fallback; device-side keys and safe logs.	Mixed decryptable/undecryptable, refresh, logs.

How it will be tested

- Four-account relationship matrix, direct RLS, two-user E2EE/private/general/group chat, reload, removed member.

Definition of Done

- Personal shows only accepted friends/groups.
- General rules are enforced.
- E2EE remains intact and failures are isolated.

ACTION 15 - Build Personal and Shared Music

Priority: P2 | Status: MISSING | Depends on: Actions 2, 6, 10, 13-14 | Unlocks: Private listening and shared music

P2

MISSING

Why now?

Music needs private storage, official provider boundaries, a proven player, and safe realtime state.

Why not earlier?

Provider/legal/storage decisions must precede UI promises.

Why not later?

A later player rewrite would disrupt Watch, room Music, and personal playback.

Current state

Music is a master-plan feature, not a complete recorded implementation.

Existing code/features to preserve

Storage abstraction, MediaStage/player patterns, cleanup route, room provider, permissions, no token leakage.

What changes

Provider adapters, upload/retention metadata, personal player, room/group Music section, queue/now-playing.

Implementation plan

- Keep R2-ready provider-neutral storage; do not implement R2 now.
- Use official provider support; explain search/metadata/redirect-only limitations.
- Delete personal uploads/metadata after 48 hours, including abandoned uploads.
- Share track references and playback state, never provider tokens/account details.

Database changes

Additive upload/provider/shared-state tables, expiry indexes, private storage policies.

Realtime changes

Room/group Music uses existing provider/event reducer; personal connections remain server-side.

UI changes

Provider list, connect/disconnect, search source selector, player, queue, room/group controls.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Token leak, unsupported playback, protected-content bypass, wrong cleanup, event drift.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
Personal uploads	Upload multiple MP3/FLAC/browser-supported files with progress, retry, cancel, metadata and artwork.	Reuse storage abstraction/media player; add upload service/UI.	Private storage; unsupported formats are clearly rejected; 48-hour retention.	Multi-file, unsupported, abandoned, 48-hour cleanup.
Provider connections	Connect Spotify, Apple Music, TIDAL, Qobuz, YouTube Music through official OAuth/API/SDK only.	Reuse SSR/server secrets and provider adapter pattern.	Tokens server-side, minimum scopes, revocation; no scraping/download bypass.	Connect/disconnect /search/playlist/token refresh.
Personal player	Play, pause, seek, volume, queue, repeat, shuffle, artwork, metadata, persistent navigation.	Reuse MediaStage/player state.	Provider limits honest; personal credentials private.	Supported/unsupported source and navigation tests.
Room/group Music	Shared track, queue, play/pause/seek and now-playing state with control permissions.	Reuse shared provider and Watch event contract.	Broadcast sanitized references only; disabled feature server-denied.	Two-user permission, reconnect, group scope.

How it will be tested

- Upload formats, expiry simulation, provider sandbox, player navigation, two-user shared Music, disabled-feature RLS.

Definition of Done

- Supported Music works without false claims.
- 48-hour cleanup is idempotent.
- No provider credential crosses users or realtime.

ACTION 16 - Add AI and Intelligence Last

Priority: P2 | Status: MISSING | Depends on: Actions 1-15 | Unlocks: Assistive intelligence without privacy shortcuts

P2

MISSING

Why now?

AI must consume trustworthy, permissioned normal data and should be the final major implementation stage.

Why not earlier?

Before normal notes, history, tasks, calls, and access controls exist, AI would invent unreliable or unsafe semantics.

Why not later?

It must not be postponed beyond the complete non-AI foundation if AI is a product differentiator.

Current state

The master plan mentions AI, summaries, meeting assistance, tasks, and memory; no complete implementation is recorded.

Existing code/features to preserve

RLS, room/group scopes, E2EE boundary, normal search/activity/notes/events/tasks, consent and notifications.

What changes

Server AI gateway, explicit opt-in, bounded retrieval, human approval, retention/provenance, erasure controls.

Implementation plan

- Build normal manual tasks, events, notes, search, history, and meeting records first.
- Use server-side retrieval that checks current authorization on every request.
- Mark output as draft/suggestion, let users approve/edit/delete, and store provenance.
- Never send private keys or decrypted E2EE content automatically; require explicit approved client-side sharing if ever designed separately.
- Protect against prompt injection, overbroad retrieval, hallucinated actions, and retention surprises.

Database changes

Additive AI job/result/consent/provenance tables with strict RLS and retention indexes.

Realtime changes

AI jobs emit scoped status/notification events only; raw prompts/private content are not broadcast.

UI changes

Opt-in controls, consent banners, draft labels, source links, delete/export, failure states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Plaintext/E2EE leakage, unconsented recording, false summaries, task side effects, vendor outage.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

Feature-by-feature scope

Feature	What / user experience	Reuse and additions	Dependencies / security	Test and Definition of Done
AI Room Assistant	User asks an assistant about authorized room context, actions, or settings.	Reuse normal search/activity/permissions; add server AI boundary.	Explicit opt-in, scope-limited data, no automatic E2EE plaintext access.	Prompt/response redaction, permission changes, opt-out.
Live AI Meeting Assistant	Opt-in assistant listens/transcribes supported live meeting input and surfaces help.	Reuse calls/voice/consent/notes; add managed AI stream.	Visible consent, participant controls, retention, regional/provider limits.	Join/leave/consent/revocation and no silent capture.
AI-assisted action items / tasks	AI suggests tasks from confirmed notes/activity; user approves edits.	Reuse manual tasks/events/notes first.	Human confirmation, owner scope, no automatic assignments.	Suggestion/accept/reject/undo and audit.
Automated room summary	AI summarizes selected room history for authorized members.	Reuse activity/search/history; add bounded summarization job.	Room membership at request and delivery; retention and prompt injection defense.	Small/large history, removed member, export.
Meeting minutes / summary	AI drafts minutes with decisions, attendees, and follow-ups.	Reuse normal notes/events/action items.	Draft label, human edit, participant privacy, no false certainty.	Generate/edit/delete and audience tests.
Room memory	Opt-in long-term room memory stores approved durable facts and decisions.	Reuse bookmarks/history/notes and retention controls.	User/room deletion, provenance, access changes, no sensitive inference.	Add/remove/search/export/erase and membership change.

How it will be tested

- Consent/revocation, room/member scope, manual approval, prompt injection, erase/export, provider outage, no-secret log tests.

Definition of Done

- AI is optional and auditable.
- No AI feature bypasses RLS or E2EE.
- Users control retention and approve side effects.

ACTION 17 - Harden Mobile, Performance, Operations, and Technical Debt

Priority: P1 | Status: PARTIAL | Depends on: Actions 1-16 | Unlocks: Usable, supportable product

P1

PARTIAL

Why now?

After flows are real, measure bottlenecks and make all critical paths usable on phones and unreliable networks.

Why not earlier?

Premature optimization or broad cleanup can destabilize the current working core.

Why not later?

Shipping the expanded product without mobile and operations creates expensive support debt.

Current state

Dynamic loading, dead writer cleanup, shared provider, current lint/type checks, and some lifecycle handling exist; mobile/performance breadth remains.

Existing code/features to preserve

Desktop behavior, current routes, providers, E2EE, lifecycle, and Action 10 cleanup.

What changes

Mobile shell/drawers, touch controls, pagination, query budgets, subscription metrics, redacted telemetry, targeted technical debt cleanup.

Implementation plan

- Measure bundles, route timings, query plans, subscription counts, uploads, and media errors.
- Paginate messages/files/history; remove unnecessary polling; load heavy features dynamically.
- Test iOS/Android browsers, keyboard/viewport, audio/upload interruptions, and Focus Lock escape.
- Delete dead code only after full reference search; keep legacy database table if compatibility requires it.

Database changes

Indexes based on query evidence; no data deletion or broad refactor.

Realtime changes

Track channel lifecycle/reconnect metrics without private payloads.

UI changes

Mobile navigation, drawers, touch targets, saved layout, accessible focus and error states.

Security considerations

Use server actions, database RLS, storage policies, and authorized realtime checks for this scope.

What could break?

Over-optimization, sensitive logs, mobile-only regressions, dead-code deletion of live path.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- Lighthouse/route timings, device matrix, subscription leak test, log audit, bundle analysis, regression suite.

Definition of Done

- Critical flows work on mobile.
- Measured performance improves.
- Logs are useful and safe.
- No live path was removed accidentally.

ACTION 18 - Verify, Deploy, and Gate Production

Priority: P0 | Status: PARTIAL | Depends on: Actions 1-17 | Unlocks: Honest launch decision

P0

PARTIAL

Why now?

Production claims require evidence from code, database, independent browsers, deployed environment, and operations.

Why not earlier?

Testing earlier actions in isolation is necessary; the final gate must wait until all dependencies are complete.

Why not later?

A launch without a gate converts unknown behavior into user impact.

Current state

TypeScript/build/lint, production smoke, critical RLS, migration alignment, chat/delete, and profile isolation were previously reported; several browser/cron/SSR/E2EE checks remain open.

Existing code/features to preserve

All existing tests, migration history, deployment workflows, disposable accounts/rooms, rollback procedures, and current production protections.

What changes

A repeatable release checklist, test evidence, deployment verification, monitoring and rollback runbook.

Implementation plan

- Run Level 1 static checks, Level 2 unit tests, Level 3 RLS, Level 4 integration, Level 5 independent two-user, Level 6 browser UX, and Level 7 production checks.
- Verify current commit is the deployed commit; confirm migrations/policies/functions/triggers live.
- Run anonymous/role matrix, Watch/seek, chat/delete/presence, file, recovery/permanent/notification, Focus Lock, SSR refresh, E2EE failure, and cron tests.
- Do not upgrade unavailable tests to pass; record exact blockers.

Database changes

No migration reset/repair unless a concrete verified discrepancy requires it; no data deletion outside disposable test scope.

Realtime changes

Verify realtime in isolated profiles and after deployment.

UI changes

Release page/status dashboard shows only evidence-backed claims.

Security considerations

Security and privacy issues remain launch blockers even if UI/build passes.

What could break?

False pass, stale deployment, unobserved cron, environment-only auth issue, cross-action regression.

How regressions will be prevented

Use the smallest safe change, preserve the current path behind a feature flag where practical, and verify old and new paths together.

Safe rollback plan

Roll back the new route/action/feature flag or use a forward-compatible migration. Never reset the database or delete established data to undo a feature.

How it will be tested

- tsc, build, lint, diff check, all tests, live probes, independent browsers, cron logs, recovery, Focus Lock, SSR, E2EE, final code search.

Definition of Done

- Tested commit is deployed.
- All critical/high gates pass or the verdict remains NO.
- Rollback and known limitations are documented.

4. SECURITY ROADMAP

Issue -> risk -> fix -> status -> verification -> production meaning

Area	Risk	Fix	Status	Verification	Gate
Auth/session	Stale SSR cookies, unsafe next redirects, duplicate identities	Existing proxy refresh, safe allowlist, Google callback, profile gate	Source partial; OAuth/refresh verification open	Login/logout, callback, refresh, protected routes	P0 until live evidence
Anonymous access	Private rooms/messages/media/storage exposed	RLS with no public private SELECT	Critical probes previously reported pass	Direct anonymous reads	Must remain closed
Membership/roles	Self-promotion, stale/former access	Immutable identity/role scope and approved-member checks	Remediation source/live reports exist	Full role matrix	P0 regression gate
Messages	Non-sender or former member edits/deletes	UPDATE RLS + sender/scope trigger; Action 5 server check	Direct checks reported pass	UI and direct mutation	P0
Storage	Wrong path or public object access	UUID path, active member/lifecycle policy	Migration/docs present; recheck canonical/live	Object reads by role/state	P0
E2EE	Keys/plaintext leak or one bad message crashes list	Device keys, allSettled, undecryptable state, redacted logs	Source done; mixed runtime open	Missing/rotated key and log audit	P1 runtime
Realtime	Unauthorized events, duplicate channels, stale state	One provider, scoped events, dedupe/reconnect	Source done/partial runtime	Two isolated profiles	P1
Lifecycle	Expired room visible, recovery bypass, permanent room expires	Server/RLS deadlines, immutable requests, idempotent cleanup	Source/direct tests; browser/cron open	Time-controlled disposable room	P0
Social/provider	Private relationship/provider data leaks	Per-user RLS, server tokens, minimum scopes	Missing product work	Four-account direct/browser matrix	P1/P2
AI	Unconsented processing or E2EE disclosure	Opt-in, bounded retrieval, draft approval, erasure	Future	Consent/scope/prov enhancement tests	P2 launch scope

5. SUPABASE, DATA LIFECYCLE, AND REALTIME PLAN

Database operating rules

- Inspect before migrating: columns, constraints, indexes, policies, functions, triggers, grants, storage, and real data shape without exposing content.
- Keep 01_schema.sql, POLICIES.md, and ordered migrations consistent. 002/003/004 are recorded remediation layers; local 005 must be checked against live state before future pushes.
- Use additive, backward-compatible migrations with explicit status checks, UUID foreign keys, unique normalized pairs, least privilege, and rollback/forward-fix plans.
- Never put cleanup/data destruction in a security migration. Never reset the linked database to solve migration history confusion.

Lifecycle	Flow	Security / data rule
Room	create -> join/approve -> active -> expired -> recovery window -> reopened or irreversible	Server/RLS checks, approved membership, no client timer authority.
Recovery	expired -> 24-hour request window -> pending -> approved/rejected	Owner/admin only; identity immutable; preserved data only.
Reopen	approval time -> reopened_until = approval + 7 days	Normal auth/membership/RLS continue; former members do not return automatically.
Permanent	member requests -> owner/admin approves -> permanent	Clear temporary expiry; no fake future date; indefinite access subject to auth/RLS.
Files	upload -> private storage -> authorized use -> expiry -> deletion	UUID path, signed access, idempotent cleanup, no resurrection.
Group conversion	eligible room -> one group -> Personal lists	Approved members and eligible content only; retain origin reference; remove public username.

Realtime contract

- One room-level provider manages Chat, Watch, Files, Study, notifications, presence, events, and shared Music. Components consume state; they do not open duplicate room channels.
- Events carry scope, event ID, actor, timestamp, type, and validated payload. Reconnect first obtains authoritative state, then subscribes. Duplicate events are harmless.
- Late joiners, expired rooms, removed members, disabled features, and browser refreshes all re-check server state. Private keys, plaintext, provider tokens, and large files stay out of events.

6. TESTING LADDER AND RELEASE GATES

Level	Name	Examples	Proves
Level 1	Static	tsc, build, lint, diff check, prohibited-pattern searches	Code consistency, not product behavior.
Level 2	Unit	Lifecycle, permissions, reducers, event dedupe, validation, decryption mapping	Small logic correctness.
Level 3	Database/RLS	Anonymous and role matrix, storage, functions, triggers, migrations	Database blocks bypasses.
Level 4	Authenticated integration	Actions/API, lifecycle, recovery, cleanup, provider mocks	App/database agreement.
Level 5	Independent realtime	Separate persistent profiles: chat, delete, Watch, seek, presence, files	Cross-user behavior without storage leakage.
Level 6	Browser UX	Loading/error/empty, Focus Lock, mobile, E2EE failure, accessibility	Recoverable user experience.
Level 7	Production	Deployed commit, live DB, cron logs, smoke, monitoring, rollback	Real environment readiness.

Release gates

Gate	Name	Required evidence
1	Secure foundation	No anonymous private data; RLS/storage matrix; no secrets/log leakage; migrations/live state match.
2	Core rooms	Create/join/leave/roles/settings/chat/delete/storage/lifecycle/navigation pass.
3	Watch	React Player v3, play/pause/seek, subtitles, late join/reconnect, two-user sync pass.
4	Collaboration	Files, Study, Focus Lock, documents, voice/screen, events, exports, mobile critical flows.
5	Social/groups	Google/profile/privacy, follows/friends, Personal/General, conversion, notifications.
6	Music/AI	Uploads/providers/shared Music; AI consent/scope/retention; legal/provider limitations.
7	Production	Tested commit deployed, cron observed, rollback/monitoring documented, all P0/P1 evidence passed.

7. MASTER ROADMAP TABLE

Order	Action	Pri	Status	Depends	Unlocks	What remains
01	Foundation	P0	PARTIAL	Current code	All actions	Evidence and 403 root cause
02	Supabase/RLS/schema	P0	PARTIAL	01	Safe data	Live/canonical reconciliation
03	Google auth/profile setup	P0	MISSING	01-02	Identity	OAuth and onboarding
04	Profiles/social graph	P1	MISSING	03	Relationships	Privacy/follow/friend RLS
05	Room identity/roles/control	P1	PARTIAL	01-02	Managed rooms	Fresh private-room matrix
06	Shared realtime	P1	PARTIAL	02/05	Multi-user features	Independent reconnect/dedupe
07	Room messaging/notifications	P1	PARTIAL	02/05/06	Rich chat	Edit, threads, PDF, notices
08	Activity/search/events	P2	MISSING	05-07	Organized rooms	History, polls, exports
09	Adaptive workspace	P1	PARTIAL	05-08	Comfortable UX	Layout/feature flags/themes
10	Watch/shared media	P1	PARTIAL	02/06/09	Synchronized media	Two-user seek proof
11	Collaboration/voice/docs	P2	MISSING	05-10	Non-AI collaboration	Voice/docs/events
12	Lifecycle/recovery/permanent	P0	PARTIAL	02/05-11	Safe retention	24h/7d/cron/notice
13	Room-to-group	P1	PARTIAL	02/05/12	Personal groups	Semantics and idempotency
14	Personal/General chat	P1	MISSING	03-04/06/13	Social inbox	E2EE-compatible conversation model
15	Music	P2	MISSING	02/06/10/13-14	Personal/shared music	Providers/uploads/retention
16	AI intelligence	P2	MISSING	01-15	Assistant/summaries/memory	Consent/provenance/erasure
17	Mobile/performance/ops	P1	PARTIAL	01-16	Reliability	Measured device/ops proof
18	Testing/deploy/release	P0	PARTIAL	01-17	Launch decision	All required evidence

8. MASTER FEATURE REGISTER

All existing major features plus all 54 newly requested features are integrated below, not appended outside the roadmap.

#	Feature	Area	Status	Stage	What remains
01	Edit/delete own messages	Room	PARTIAL	07	Message types, sender/update RLS, propagation.
02	Shared PDFs	Room	MISSING	07/09	Private storage, viewer, notices.
03	Invite link + unique code	Room	PARTIAL	05	Validate formats and private approval.
04	Live shared notes	Collab	MISSING	06/11	Versioning and conflict handling.
05	Timer-created owner notification	Room	MISSING	07/11	Study event to notification center.
06	Member 3-dot actions	Room/Social	MISSING	04/05/14	Profile/friend/DM actions with RLS.
07	Ghost Mode	Room	MISSING	05/06	Scoped alias, moderation, privacy.
08	Voice chat + owner mute	Collab	MISSING	06/11	Managed SFU and consent.
09	Room Control Center	Room	PARTIAL	05	Unified settings and audit.
10	Room Roles & Permissions	Room	PARTIAL	02/05	Granular server/RLS enforcement.
11	Announcements	Room	MISSING	07	Manager-authored notices.
12	Pinned Messages	Room	MISSING	07	Scoped pin relation.
13	Replies / Threads	Room	MISSING	07	Parent/thread model.
14	Mentions	Room	MISSING	04/07	Member-safe mention notices.
15	Notification Center	Room/Social	PARTIAL	07/12	Per-user, scoped read state.
16	Room Search	Room	MISSING	08	Indexed private search.
17	Activity / Room History	Room	MISSING	08	Timeline and retention.
18	Custom Layout Presets	UX	MISSING	09	User/room saved layout.
19	Owner-controlled layout	UX	MISSING	05/09	Server feature visibility.
20	Room Themes / Appearance	UX	MISSING	09	Validated contrast-safe themes.
21	Polls & Voting	Collab	MISSING	06/08	One-vote RLS and live results.
22	Shared Queue	Watch/Music	MISSING	10/15	Permissioned queue state.
23	Now Playing / Shared State Bar	Watch/Music	MISSING	10/15	Sanitized shared state.
24	User Presence States	Realtime	PARTIAL	06	Online/activity/away states.
25	Leave / Kick / Ban / Block	Room	PARTIAL	05	Membership and moderation actions.
26	Screen Sharing	Collab	MISSING	06/11	Consent and managed media.
27	Raise Hand	Collab	MISSING	06/11	Ephemeral activity signal.
28	Room Rules & Welcome	Room	MISSING	05/08	Join and dashboard content.
29	Temporary Guest Access	Lifecycle	MISSING	05/12	Bounded guest session.
30	Room Lock	Lifecycle	MISSING	05/12	Server-enforced join lock.
31	Scheduled Rooms	Lifecycle	MISSING	08/12	Cron/timezone-safe schedule.
32	Room Events	Collab	MISSING	08/11	Event records/reminders.
33	Shared Whiteboard Cursor Presence	Collab	MISSING	06/11	Rate-limited ephemeral cursors.
34	Collaborative Document Editing	Collab	MISSING	06/11	Versioned document state.
35	Room Bookmarks / Saved Content	Room	MISSING	08	User-scoped saved references.
36	Room-Specific User Profiles	Room/Social	MISSING	04/05/08	Scoped alias and privacy.
37	Smart Room Search & Filters	Room	MISSING	08	Normal indexed filters before AI.

#	Feature	Area	Status	Stage	What remains
38	Room Backup / Export	Room	MISSING	08/12	Signed bounded archive.
39	Shared Countdown / Event Timer	Collab	MISSING	06/08	Server-time synchronization.
40	Live Activity Bar	Realtime	MISSING	06/08	Ephemeral room activities.
41	Room Dashboard / Home	Room	PARTIAL	05/08/09	Lifecycle-aware room landing.
42	Room Clips / Highlights	Watch	MISSING	10	Bounded timestamped references.
43	Smart Auto-Organization	Room	MISSING	08	Deterministic suggestions first.
44	Room Templates	Room	MISSING	05/08	Settings-only reusable templates.
45	Guest / Temporary Identity Mode	Room	MISSING	05/12	Scoped temporary identity.
46	Room Reputation / Trust Levels	Moderation	MISSING	08/11	Transparent, appealable signals.
47	Smart Notification Controls / DND	Notifications	MISSING	07	Per-type duration/never controls.
48	Cross-Room Personal Dashboard	Dashboard	MISSING	08/14	Only joined-room data.
49	AI Room Assistant	AI	MISSING	16	Opt-in authorized retrieval.
50	Live AI Meeting Assistant	AI	MISSING	11/16	Consent and live input boundary.
51	AI-assisted Action Items / Tasks	AI	MISSING	08/11/16	Draft and human approval.
52	Automated Room Summary	AI	MISSING	08/16	Bounded authorized history.
53	Meeting Minutes / Meeting Summary	AI	MISSING	11/16	Draft, edit, provenance.
54	Room Memory	AI	MISSING	08/12/16	Opt-in durable facts and erasure.

9. NEXT 10 ACTIONS

Immediate work based on the latest recorded remediation state.

#	Action	Why now	Expected result
1	Resolve product decisions	Approve Google-only/legacy login, profile fields, group feature surface, username behavior, Music provider limits, AI consent.	Signed decision record before new schema.
2	Reproduce the room-creation 403	Trace auth/session/action/RPC/RLS with disposable accounts.	Fresh private-room creation and isolation matrix.
3	Reconcile live/canonical Supabase	Recheck 002-005, policies, functions, triggers, schema docs.	No migration or policy drift.
4	Implement Google OAuth/onboarding	Callback, setup gate, safe redirect, dashboard configuration.	Stable identity for new/existing users.
5	Implement profiles/privacy/social graph	Profile setup, posts, follows, friends, RLS.	Public/private relationship matrix.
6	Finish room permissions and Control Center	Leave, granular admin roles, features, invites, lock, rules.	Trusted room management.
7	Complete shared realtime and Watch proof	A/B play, pause, seek, presence, reconnect, late join.	Independent two-user evidence.
8	Finish lifecycle browser/cron evidence	24-hour recovery, 7-day reopen, permanent conversion, notifications.	Safe expiry and recovery proof.
9	Build Personal/General/groups	Conversation model, requests, group chat/workspace decision.	Separate social messaging without E2EE regression.
10	Build Files/Music and mobile release ladder	Private uploads, provider adapters, shared Music, performance/mobile tests.	Expand only after storage and operations are safe.

10. FINAL PRODUCTION-READY CHECKLIST

PRODUCT

- ☐ Core auth/profile flow works
- ☐ Rooms create/join/leave
- ☐ Membership/roles/settings
- ☐ Room chat/edit/delete/threads/mentions
- ☐ Watch/play/pause/seek
- ☐ Files/PDF/study/Focus Lock
- ☐ Personal/General/groups
- ☐ Music scope and provider limits honest
- ☐ AI is opt-in and not required for core use

SECURITY

- ☐ No anonymous private data
- ☐ RLS direct matrix passes
- ☐ Former/pending/rejected/unrelated users denied
- ☐ Storage path/lifecycle protected
- ☐ No key/plaintext/token leakage
- ☐ Realtime events scoped
- ☐ Recovery/permanent approvals server and DB enforced

DATABASE

- ☐ Local/live migrations match
- ☐ Canonical schema/docs are current
- ☐ Indexes/constraints verified
- ☐ Expiry/recovery/permanent state verified
- ☐ Cleanup idempotent
- ☐ No destructive migration surprise
- ☐ Destroyed data is never restored

REALTIME

- ☐ Independent browser profiles proven
- ☐ Chat/delete/presence pass
- ☐ Watch/seek pass both directions
- ☐ Reconnect/late join recover state
- ☐ No duplicate channels
- ☐ Feature disable/removal cleans subscriptions

UX / MOBILE

- ☐ Loading/empty/error/denied states
- ☐ Center chat/right feature layout
- ☐ No blank/overlapping panels
- ☐ Keyboard/touch/responsive
- ☐ Focus Lock escape
- ☐ Accessible contrast and labels
- ☐ DND/notification controls understandable

OPERATIONS

- ☐ Tested commit deployed
- ☐ Vercel smoke pass
- ☐ Cron execution observed
- ☐ Monitoring is redacted
- ☐ Rollback runbook exists
- ☐ TypeScript/build/lint/diff pass
- ☐ Known limitations documented

FINAL VERDICT

PRODUCTION READY: NO for the complete OmniLume vision at the latest recorded state. The existing room/security foundation is substantially remediated, but the roadmap still has open P0/P1 verification gates and large missing product areas: Google onboarding, profiles/social relationships, Personal/General chat, group semantics, Music, broader collaboration, and AI.

11. IMPLEMENTATION PRINCIPLES AND HANDOFF

- Protect the existing codebase: extend first, improve second, refactor only when needed, rewrite only when absolutely necessary.
- Every major action has a safe change plan: preserve existing code/features, make the smallest change, identify breakage, test old and new paths, and roll back forward without resetting data.
- Every feature must be described at the correct evidence level. Implemented in source is not runtime verified; runtime verified is not production verified.
- Normal non-AI systems come first: manual tasks, events, notes, search, history, activity, and meeting records. AI comes only after those systems and their permissions are proven.
- Rooms remain separate from Personal Chat. Conversion is explicit, idempotent, privacy-preserving, and must not silently create a public group link.
- The release decision stays NO until critical security issues are absent and required browser, database, lifecycle, cron, session, and deployment evidence is actually observed.

Primary repository references

Area	References to inspect and preserve
Routes	src/app/page.tsx, login, explore, create-room, room/[id], messages, home, room settings, cleanup route
Actions	src/actions/chat.ts, rooms.ts, recovery.ts, notifications.ts, media.ts, members.ts, study.ts
Room UI	RoomRealtimeProvider.tsx, RoomChat.tsx, MediaStage.tsx, FilesTab.tsx, StudyStage.tsx, MembersTab.tsx, RoomNotifications.tsx
Hooks/libs	useRoomSync.ts, useRoomPresence.ts, usePrivateChat.ts, encryption.ts, focus-lock.ts, room-lifecycle.ts, storage.ts
Database	supabase/01_schema.sql, POLICIES.md, migrations 002-005, local CLI metadata
Planning	OMNILUME_MASTER_PLAN.md plus remediation and verification history from this conversation

End of replacement blueprint